TECHNICAL REPORT | SYSTEMS ENGINEERING & VIRTUALIZATION

Linux Desktop Performance Evaluation and Optimization

A Comparative Benchmark between GNOME and XFCE in Virtualized Development Workstations

Davide Torelli

Author & Project Lead

CYBERSECURITY MASTER STUDENT

AI GOVERNANCE & CLOUD COMPLIANCE

Experimental Testbed Specifications		
Operating System	Ubuntu 24.04.4 LTS (`x86_64`)	Linux Kernel `6.17.0-22-generic`
Hypervisor Platform	Oracle VM VirtualBox (4 vCPUs, 4 GB RAM)	VBoxSVGA Controller (3D Accel. Disabled)
Evaluated Environments	Ubuntu Default (GNOME 46 + Mutter)	Optimized Stack (XFCE 4.18 + LightDM)
Telemetry & Tooling	Memory profiling via `procps` (`free -m`)	Multi-core CPU sampling via `sysstat` (`sar -u 1 5`)

Abstract

This report documents a hands-on performance comparison between Ubuntu 24.04.4 LTS with GNOME 46 and an optimized XFCE 4.18 stack running on LightDM, both deployed on the same VirtualBox virtual machine. The goal was to quantify how much a lightweight desktop environment actually saves in terms of RAM and CPU overhead on a resource-constrained development workstation.

Memory and CPU telemetry was collected via `free -m` and `sar -u` across idle and active workload profiles. The results show a **26.2%** reduction in idle RAM usage and a drop in kernel-space CPU overhead (`%system`) from **13.21%** to **5.57%** during active IDE sessions. Available processor headroom (`%idle`) rises from **57.13%** to **87.80%**, and cold-boot time is **22.4%** faster under XFCE. The report also covers a framebuffer deadlock (`drm_fb_helper_damage_work`) encountered during initial setup and the mitigations applied to stabilize the environment.

Keywords: System Benchmarking, XFCE, GNOME Shell, Linux Kernel Telemetry, Virtualization Overhead, Workstation Optimization.

1. Architectural Framework and System Specifications

Before running any benchmark, it is worth clarifying what XFCE actually is from a systems perspective, since the term gets used loosely. In technical literature, XFCE is a **Desktop Environment (DE)**: a modular collection of user-space components that includes a window manager (`xfwm4`), a panel and desktop daemon (`xfce4-panel`), a file manager (`thunar`), and a set of lightweight session services. It falls into the category of **Lightweight Desktop Environments**, characterized by minimal GTK dependencies, no background indexing daemons, and a deliberately low CPU and RAM footprint.

The **Display Manager (DM)** is a separate component — in this setup, `lightdm` paired with `lightdm-gtk-greeter` — responsible for spawning the X server and handling user authentication, decoupled from the desktop session itself.

1.1 Experimental Testbed and Allocated Resources

All tests were run inside a VirtualBox virtual machine configured to simulate a mid-range developer workstation. The goal was to keep the environment as reproducible as possible across runs, so hardware parameters were fixed and host background activity was minimized during telemetry collection.

Table 1 summarizes the full hardware and software configuration:

Configuration Parameter	Assigned Specification / Implementation Detail
Operating System	Ubuntu 24.04.4 LTS (`x86_64`) | Linux Kernel `6.17.0-22-generic`
Hypervisor	Oracle VM VirtualBox (v7.x architecture)
Virtual Processor (vCPU)	4 dedicated vCPUs (unpinned host scheduling)
System Memory (RAM)	**4,096 MB** nominal (**3,914 MB** addressable by kernel)
Graphics Subsystem	`VBoxSVGA` / `VMSVGA` adapter (Hardware 3D disabled)
Target Desktop Stack	`xfce4`, `xfce4-goodies`, `lightdm`, `lightdm-gtk-greeter`
Hypervisor Guest Modules	`virtualbox-guest-x11`, `virtualbox-guest-utils`
Telemetry Tooling	`procps` (`free -m`), `sysstat` (`sar -u 1 5`), `systemd-analyze`

Table 1: Experimental testbed specifications and telemetry tooling baseline.

1.2 Software Stack and Telemetry Tooling

XFCE was installed on top of a minimal Ubuntu base via standard repositories, without any third-party PPAs. Three tools handled all telemetry collection:

- **Memory Sampling**: `free -m` from `procps` gave a straightforward snapshot of RAM usage, separating process memory from kernel buffers and page cache.
- **CPU Workload Profiling**: `sar -u 1 5` from `sysstat` sampled all four vCPUs at one-second intervals, breaking down time spent in user space (`%user`), kernel space (`%system`), and idle (`%idle`).
- **Boot Timeline Inspection**: `systemd-analyze` measured the kernel and userspace initialization time on each cold boot, useful to verify that the Plymouth mitigations (Section 2) were actually working.

2. Boot Deadlock Mitigation and Kernel Stabilization

Setting up GNOME on this VirtualBox configuration was not straightforward. During early testing with kernel `6.17.0-22-generic`, the system consistently failed to reach the graphical login screen: the kernel worker thread `drm_fb_helper_damage_work` pinned one vCPU at 100%, causing a soft lockup that blocked LightDM from starting. After some investigation, the root cause turned out to be a race condition on the virtual framebuffer, triggered by three components initializing concurrently: the `VBoxSVGA` display driver, Plymouth's animated splash screen, and LightDM's X server.

The fix required four targeted interventions, summarized in Table 2:

Mitigation Layer	Target Component	Applied Configuration / Directive
Early Boot Splash	Systemd units	`systemctl mask plymouth-start.service ...`
Kernel Command Line	GRUB loader	Stripped `splash` flag; enabled serial fallback
Hypervisor Constraints	Oracle VirtualBox	Disabled 3D acceleration; allocated 4 pinned vCPUs
Display Management	LightDM & `xfwm4`	`logind-check-graphical=true`; compositor off

Table 2: Systemic mitigation matrix for hypervisor DRM framebuffer stabilization.

2.1 Masking Plymouth Splash Services

Plymouth was the main bottleneck: it holds the framebuffer during the kernel-to-userspace handoff, and in this virtualized setup it clashed directly with LightDM's X server startup. Masking the relevant systemd units removes Plymouth from the boot sequence entirely, forcing a clean text-mode transition into the display manager:

```
sudo systemctl mask plymouth-start.service plymouth-quit-wait.service
```

Masking works by symlinking the service files to `/dev/null`, which evicts them from systemd's dependency graph. This eliminates the framebuffer contention without requiring kernel patches or custom initramfs configurations.

2.2 Kernel Command-Line Sanitization in GRUB

In `/etc/default/grub`, the `splash` flag was removed from the kernel command line:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet"
```

Running `sudo update-grub` applies the change. If the lockup persists in other virtualized setups, adding `nomodeset` to the same line disables kernel mode-setting entirely, bypassing the DRM bridge that causes the contention.

2.3 Hypervisor Parameterization and Hardware Constraints

Two changes were made directly in VirtualBox settings:

- **Hardware 3D Acceleration**: Disabled in the display settings. On recent Linux kernels, the `VBoxSVGA` 3D passthrough path triggers buffer allocation failures in the virtualized DRM memory space, which is what causes the lockup.
- **vCPU Allocation**: Fixed at 4 vCPUs to prevent kernel worker threads from starving the scheduler when running on a single core.

2.4 Display Manager Synchronization and Compositor Disabling in xfwm4

To prevent LightDM from initializing the greeter before the virtual graphics pipeline is ready, the following option was added to the LightDM seat configuration:

```
logind-check-graphical=true
```

Finally, compositing was disabled in `xfwm4` via *Settings* → *Window Manager Tweaks* → *Compositor*. Turning off shadows, opacity and blur removes the CPU-side rendering pipeline entirely, which is both a stability fix and a performance gain.

3. Empirical Evaluation and Benchmark Telemetry

The benchmark compared a stock Ubuntu 24.04.4 LTS installation with GNOME 46 and Mutter against an XFCE 4.18 stack running on LightDM, both on the same VirtualBox VM described in Section 1. Two workload profiles were tested:

- **Idle Baseline**: measured 120 seconds after login, with only a terminal open for telemetry collection. This gives the desktop environment time to settle — session daemons and autostart services finish initializing, and memory usage stabilizes.
- **Active Development Workload (IDE + Tools)**: Visual Studio Code with LSP workers actively indexing a multi-module project, two Bash terminals, and a background static linter running concurrently.

Each profile was measured across three independent cold-boot cycles to reduce single-run noise. The numbers reported here are steady-state snapshots; inter-run memory variance stayed within ±15 MB across all iterations.

3.1 Volatile Memory Consumption and Headroom

Table 3 shows RAM usage from `free -m` under both profiles. The difference is most visible at idle, where XFCE's lower daemon count keeps memory usage well below GNOME's baseline.

Workload Scenario	GNOME 46	XFCE 4.18	Net Delta	Relative Variation
Idle Baseline	**1,266 MB**	**934 MB**	**-332 MB**	**-26.2%** Memory Used
Active Workload (IDE + Tools)	**2,464 MB**	**2,200 MB**	**-264 MB**	**-10.7%** Memory Used
Available Headroom (Active)	**1,449 MB**	**1,713 MB**	**+264 MB**	**+18.2%** Operational Margin

Table 3: Physical memory utilization sampled via `free -m` (steady-state snapshot, $N = 3$ cold-boot iterations, variance ±15 MB).

The **264 MB** headroom gain under active load matters in practice: on a 4 GB VM, that is the difference between LSP workers running smoothly and the system starting to pressure swap. GNOME's higher baseline comes primarily from GNOME Shell, tracker-miner, and the Mutter compositor process, none of which are present in the XFCE stack.

3.2 Virtual Processor Core Workload Distribution

Table 4 shows CPU utilization from `sar -u 1 5` under both profiles. The gap between the two environments is larger here than in RAM, especially under active load.

3.3 Boot Timeline Comparison

Table 5 shows cold-boot times from `systemd-analyze`. Both the kernel and userspace phases are faster under XFCE, which reflects fewer services starting up and the absence of Plymouth contention after masking.

3.4 Telemetry Insights

Three observations worth highlighting from the data:

1. **Kernel-space overhead under load**: The `%system` spike to **13.21%** under GNOME is almost entirely attributable to Mutter. In a VirtualBox environment with 3D acceleration disabled, Mutter falls back to `llvmpipe` software rendering, which generates a high

CPU Metric	GNOME 46	XFCE 4.18	Net Delta	Functional Impact
Idle %system	4.24%	0.62%	-3.62 pp	Lower resident compositor overhead
Idle %idle	87.98%	99.12%	+11.14 pp	Greater headroom at rest
Active %system	13.21%	5.57%	-7.64 pp	$>57\%$ reduction in kernel overhead
Active %user + %system	23.10%	11.87%	-11.23 pp	Substantial drop in total CPU load
Active %idle	57.13%	87.80%	+30.67 pp	Maximum cycles for developer toolchains

Table 4: vCPU core utilization sampled via `sar -u 1 5` (5-second window, steady-state load).

Boot Phase	GNOME 46	XFCE 4.18	Net Delta	Impact
Kernel initialization	8.925 s	6.581 s	-2.344 s	Faster DRM handoff
Userspace initialization	31.325 s	24.634 s	-6.691 s	Fewer session daemons
Total boot time	**40.250 s**	**31.216 s**	**-9.034 s**	**-22.4%** faster to graphical target

Table 5: Cold-boot timeline via `systemd-analyze` (single representative run per environment).

volume of kernel interrupts per frame. XFCE without compositing avoids this path entirely, keeping `%system` at **5.57%**.

2. **CPU headroom at 87.80% idle**: Under active load, XFCE leaves nearly 9 out of 10 CPU cycles free for actual work. On GNOME, the compositor alone consumes enough cycles to bring that headroom down to **57.13%** — a **30.67 pp** difference that directly affects compile times and LSP responsiveness.

3. **Memory stability across runs**: XFCE's idle footprint stayed between **920 MB** and **934 MB** across all three boot cycles ($\Delta = 14$ MB peak-to-peak). There were no unexpected daemon spikes between runs, which makes the environment predictable for benchmarking and for daily use on constrained hardware.

4. Methodological Limitations and Architectural Trade-offs

The results in Section 3 are solid within the scope of this setup, but there are a few constraints worth being explicit about before drawing broader conclusions.

4.1 Threats to Validity and Experimental Constraints

1. **Unpinned host CPU scheduling**: The 4 guest vCPUs run under the host OS scheduler without CPU pinning. Host background activity was kept minimal during telemetry windows, but small amounts of scheduling jitter and L3 cache pressure from the host cannot be fully excluded. Eliminating this would require either bare-metal CPU pinning via `isolcpus` or a Type-1 hypervisor like KVM/QEMU with dedicated cores.

2. **Software-rendered graphics**: Hardware 3D acceleration was disabled to prevent the `VBoxSVGA` DRM lockup described in Section 2. As a result, Mutter ran on `llvmpipe` — a CPU-bound software renderer — which significantly amplifies its `%system` footprint. The **13.21%** kernel-space overhead recorded in Table 4 is a worst-case figure for GNOME in this configuration. On bare metal with a real GPU, compositing overhead would be substantially lower.

3. **Short telemetry windows**: `sar -u 1 5` captures 5-second snapshots, which are sufficient for measuring steady-state compositor and daemon overhead but not for longer workloads. Extended profiling — for example, a 60-minute continuous build — would be needed to evaluate thermal throttling and swap pressure under sustained memory load.

4.2 Architectural Trade-offs of Non-Composited Environments

Dropping the compositor and switching to a non-composited X11 session comes with real functional trade-offs, summarized in Table 6.

Subsystem Feature	GNOME 46 (Composited)	XFCE 4.18 (Non-Composited)
Display Server Protocol	Wayland / XWayland hybrid	Native X11 (`Xorg`)
VSync & Tearing Mitigation	Enforced via Mutter pipeline	Disabled; potential screen tearing on rapid redraws
HiDPI & Fractional Scaling	Native per-monitor scaling	Limited; relies on X11 global DPI / font scaling
Global File Indexing	Automated (`tracker-miner-fs`)	Manual / On-demand (zero background I/O tax)

Table 6: Functional trade-offs between composited and non-composited desktop stacks.

The most noticeable trade-off in daily use is screen tearing during fast scrolling or video playback, since there is no VSync buffer without a compositor. HiDPI limitations are less relevant in a VM context, where the virtual display typically runs at a fixed resolution. The absence of `tracker-miner-fs` is, in practice, a net positive for a development workstation — background filesystem indexing adds I/O noise without providing much benefit in a terminal-heavy workflow.

5. Conclusions and Engineering Assessment

The benchmark confirms what was expected going in, but with concrete numbers attached. Running XFCE 4.18 with compositing disabled on Ubuntu 24.04.4 LTS (kernel `6.17.0-22-generic`) produces measurable gains across every metric tested:

- **Boot time**: Plymouth masking and LightDM synchronization reduce cold-boot time from **40.250 s** to **31.216 s**, a **22.4%** improvement (Table 5). The system reaches the graphical login screen consistently and without the DRM lockups (`drm_fb_helper_damage_work`) encountered during initial setup.
- **Memory**: At idle, XFCE uses **332 MB** less RAM than GNOME (**-26.2%**). Under active development load, the gap narrows to **264 MB** (**-10.7%**), but the available headroom is still **18.2%** larger (Table 3). On a 4 GB VM, that margin is meaningful when running LSP workers, a linter, and a couple of terminals simultaneously.
- **CPU**: Kernel-space overhead drops from **13.21%** to **5.57%** under load, and idle CPU headroom rises from **57.13%** to **87.80%** (Table 4). Most of the GNOME overhead comes from Mutter's compositing pipeline running on `llvmpipe` in a software-rendered VirtualBox environment — a known worst-case scenario for composited desktops.

The trade-offs are real: no VSync, limited HiDPI support, no Wayland. But for a development workstation running inside a VM on mid-range hardware, those are acceptable compromises. The methodology and raw telemetry in this report are fully reproducible on any VirtualBox setup with the same configuration.